1

Introduction

1.1 Background and Context:

The increasing reliance on digital devices and the need for efficient organisation and management of information have led to a growing demand for note-taking applications. These apps enable users to capture, organise, and access their thoughts, ideas, and important information conveniently and effectively. In response to this demand, we undertook the development of a note-taking app to provide users with a streamlined and intuitive platform for managing their notes.

1.2 Project Objectives:

The primary objective of our project was to design and develop a note-taking app that offers a comprehensive set of features and a user-friendly interface. Our aim was to create an app that would enhance productivity, facilitate seamless note-taking, and improve the overall note management experience for users.

1.3 Scope of the Project:

The scope of our project encompassed the conceptualization, design, and implementation of the note-taking app. We focused on key functionalities such as note creation, editing, organisation. Additionally, we integrated cloud-based storage and synchronisation to ensure that users can access their notes from multiple devices and maintain data consistency.

1.4 Methodology/Approach:

Our project followed an agile development methodology, allowing for iterative development and continuous feedback. We employed the Flutter framework, which provides a cross-platform development environment, to build the app's frontend. Firebase, a powerful backend platform, was chosen for its scalability and integration capabilities, enabling us to incorporate essential features such as user authentication and real-time data synchronisation.

Through the diligent application of our methodology and the utilisation of modern development tools and technologies, we aimed to deliver a robust and user-centric note-taking app that meets the needs of individuals seeking an efficient and reliable solution for capturing and managing their digital notes.

2.1 User Requirement

1. User Requirement: ***Account Creation and Authentication***
   Description: The note-taking app should support user account creation and authentication to ensure data privacy and personalised user experience.
   Importance: High
   Source: User interviews and market research

2. User Requirement: ***Note Creation and Editing***
   Description: Users should be able to create, edit, and delete notes easily within the app.
   Importance: Essential
   Source: User feedback and usability testing

3. User Requirement: ***Switching account***
   Description: User should be able to switch account
   Importance: Medium
   Source: User surveys and competitor analysis

2.2 User Stories and Cases

User Stories:

1. As a student, I want to be able to create and organise notes for different subjects, so I can keep track of my study materials effectively.
2. As a user, I want to be able to access my notes from multiple devices, so I can view and edit them seamlessly on my phone, tablet, or computer.
3. As a user, I want a user-friendly interface with intuitive controls, so I can quickly create, edit, and delete notes without any hassle.

Use Cases:

1. **Create Note:**
   Description: The user creates a new note with a title and content.
   Actors: User
   Preconditions: User is logged into the app.
   Main Flow:
   User selects the option to create a new note.
   User enters the title and content of the note.
   User saves the note.
   Postconditions: The new note is created and saved in the user's notes collection.

2. **Edit Note:**
   Description: The user edits an existing note.
   Actors: User
   Preconditions: User is logged into the app and has at least one note.

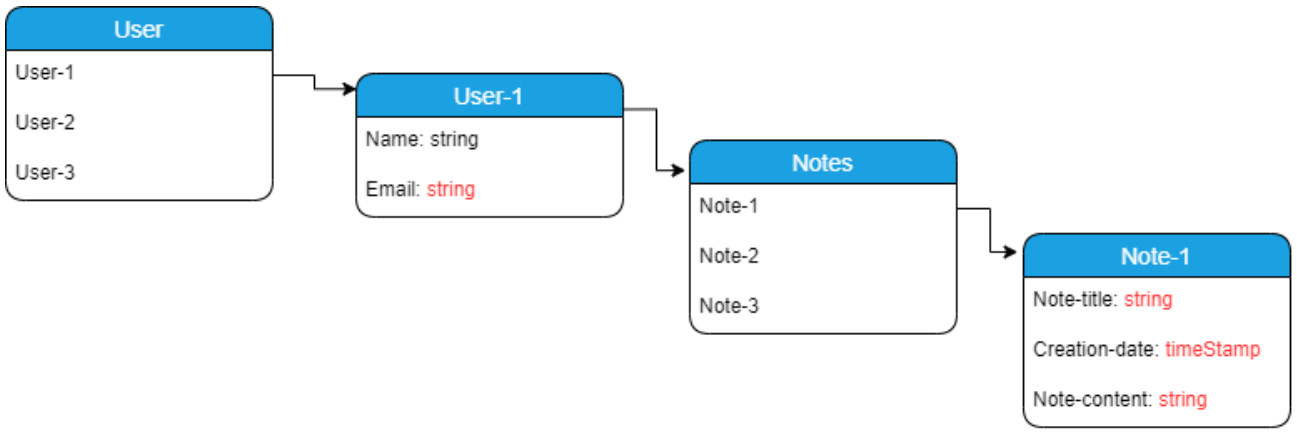
Main Flow:
User selects the note they want to edit.
User modifies the title and/or content of the note.
User saves the changes.
Postconditions: The selected note is updated with the edited information.

3. **Switch Account:**
   Description: The user want to keep separate Account one for his college and one for his personal use
   Actors: User
   Preconditions: Must Have an account to switch to
   Main Flow:
   User Click on his profile which trigger a switch account pop up screen. Through which they can go to their another account
   Postconditions: The selected account is then opened showing all the notes in that account.

2.3 Architectural Design

2.3.1 Database Structure

- Collection: User
  - Document (auto-generated ID):
    - Field: name (string)
    - Field: email (string)
    - Collection: Notes
      - Document (auto-generated ID):
      - Field: creation_date (timestamp or date/time format)
      - Field: note_content (string or text)
      - Field: note_title (string)



So, we have a collection called "**User**" that stores information about individual users. Each user document within the "User" collection has fields to store the **user's name** and **email address**. Additionally, every user document has a subcollection called**"Notes"** which contains documents representing the notes created by that user. Each note document has fields to store the creation date, the content of the note, and its title.

In terms of security rules, the rules we have defined allow authenticated users to read, delete, and update notes within the "Users" collection, specifically under the "Notes" subcollection.

```
service cloud.firestore {
  match /databases/{database}/documents {
        match /{document=**} {
        allow write
        }


    match /Users/{userId}/Notes/{noteId} {


      allow read, delete, update: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

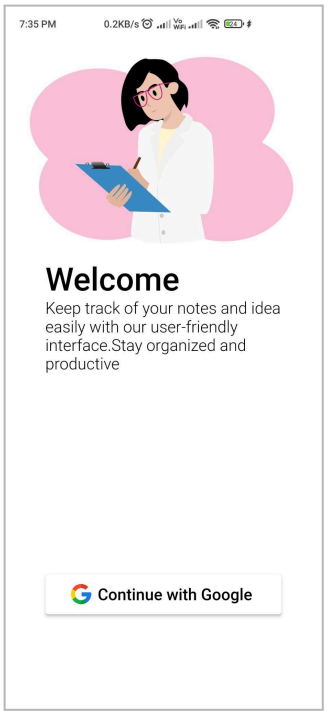
These rules ensure that only authenticated users who own the note (identified by their user ID) can perform these actions, providing security and privacy for our app.

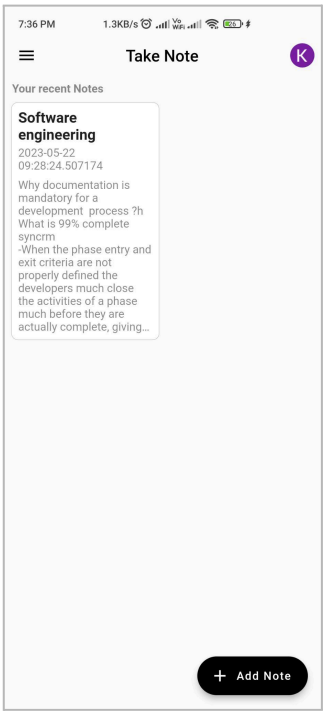
To enable user authentication, we are utilising Firebase Google Authentication, which allows users to sign in to our app using their Google accounts. This ensures that only authorised users can access the app's features and data.

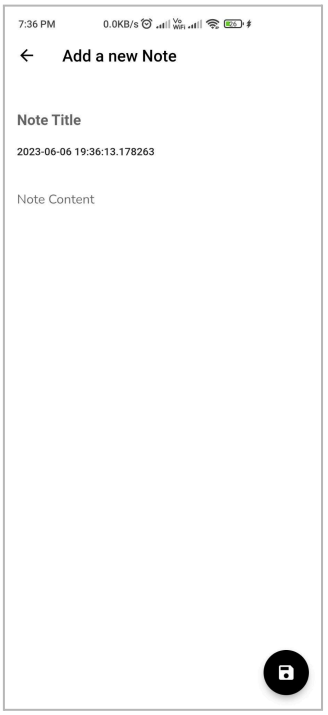
2.3.2 Interface

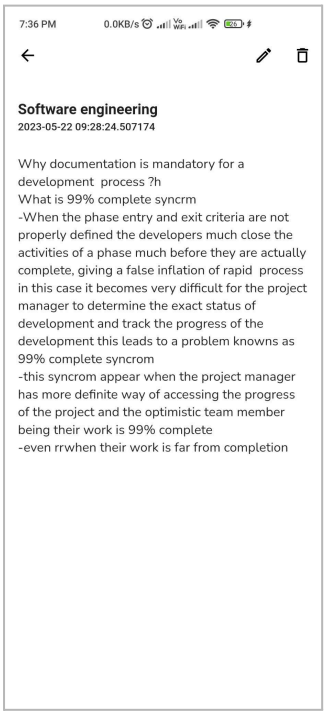
In our note-taking app, we have identified the following interfaces that the app interacts with:

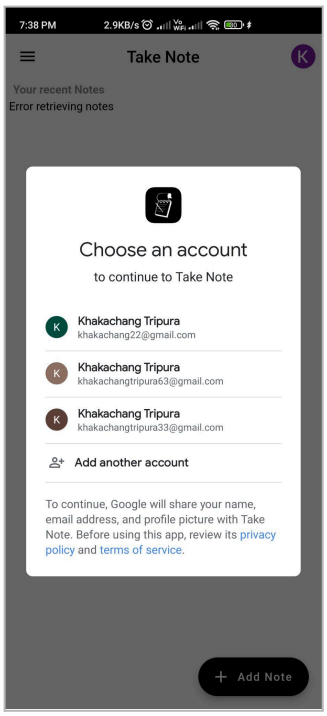
Internal Interfaces:
User Interface (UI): The user interface is the primary interface through which users interact with the app. It includes a login screen, Notes collection view screen, create note button,Save button,Particular note view screen, edit button, delete button and account switching pop up screen.











External Interfaces:
Firebase Firestore API: We integrate with the Firebase Firestore API to store and retrieve user data, including user information and notes. This API allows us to perform CRUD operations on the database and ensures data synchronisation across different devices.

2.3.3 Components:

Our note-taking app consists of the following major components:

Sign In Component:
Responsibilities: Handles user registration, authentication, and management.
Functionality: Allows users to sign up, sign in, and manage their account.

Home Screen Component:
Responsibilities: Main page of the app which allows users to view their note collection.
Functionality: Include many other components through which users can perform tasks.

Note Card Component:
Responsibilities: Display notes in home screen as a grid view with note title and note content
Functionality: Display note in homescreen in grid view with auto height adjustment depending on the note content.

Note editor Component:
Responsibilities: Handles note creation.
Functionality: Provide functionality for creating notes

Note Reader Component:
Responsibilities: Handles Note view, note deletion.
Functionality: Provide functionality to view note and delete note

Note Update Component:
Responsibilities: Handles editing existing notes.
Functionality: Provide functionality to edit existing notes.

App Style Component:
Responsibilities: Provide style for the note.
Functionality: Overall style of the notes including font family and colour theme for the app.

3.1 Flutter Framework

The note-taking app is developed using the Flutter framework. Below are some key aspects of the implementation:

User Interface Design:
We utilised Flutter's widget-based approach to build the app's user interface. Here's an example of a Flutter widget tree representing the layout of a note editing screen:

```
children: [
  SizedBox(
    height: 28.0,
  ), // SizedBox
  TextField(
    controller: _titleController,
    decoration: InputDecoration(
      border: InputBorder.none,
      hintText: 'Note Title',
    ), // InputDecoration
    style: AppStyle.mainTitle,
  ), // TextField
  SizedBox(
    height: 8.0,
  ), // SizedBox
  Text(
    date,
    style: AppStyle.dateTitle,
  ), // Text
  SizedBox(
    height: 28.0,
  ), // SizedBox
  TextField(
    controller: _mainController,
    keyboardType: TextInputType.multiline,
    maxLines: null,
    decoration: InputDecoration(
      border: InputBorder.none,
      hintText: 'Note Content',
    ), // InputDecoration
    style: AppStyle.mainContent,
  ), // TextField
],
```

3.2 Firebase Integration

Firebase services are integrated into the app to provide various functionalities. Here's an example of Firebase integration for user authentication using Firebase Authentication and saving notes to Firestore:

3.2.1 Firebase Authentication

```
child: ElevatedButton.icon(
  style: ElevatedButton.styleFrom(
      elevation: 3,
      backgroundColor: Colors.white,
      foregroundColor: Colors.black,
      minimumSize: Size(double.infinity, 50)),
  onPressed: () {
    final provider =
        Provider.of<GoogleSignInProvider>(context, listen: false);
    provider.googleLogin(context);
  },
  icon: Image.asset(
    "images/google.png",
    width: 25,
    height: 25,
  ), // Image.asset
  label: Text(
    "Continue with Google",
    style: TextStyle(
      fontSize: 20,
    ), // TextStyle
  ), // Text
), // ElevatedButton.icon
```

3.2.2 Firebase Firestore

```
//Save Botton
floatingActionButton: FloatingActionButton(
  backgroundColor: Colors.black,
  onPressed: () async {
    User? user = FirebaseAuth.instance.currentUser;
    if (user != null) {
      try {
        String userId = user.uid;
        DocumentReference userRef =
            FirebaseFirestore.instance.collection("Users").do

        // Create or update the user document with relevant d
        await userRef.set({
          "name": user.displayName,
          "email": user.email,

          // Add any other relevant user data
        });

        // Create the note document under the user document
        DocumentReference noteRef = userRef.collection("Notes
        await noteRef.set({
          "note_title": _titleController.text,
          "creation_date": date,
          "note_content": _mainController.text,
        });

        print(noteRef.id);
        Navigator.pop(context);
      } catch (e) {
        print("Error writing note: $e");
      }
    } else {
      print("User not authenticated");
    }
  },
  child: Icon(Icons.save),
), // FloatingActionButton
```

3.3 Challenges and Solutions

Throughout the implementation process, we faced various challenges and found solutions to overcome them. Here are a few examples:

Challenge: Handling User Authentication Errors:
Solution: We implemented error handling mechanisms in the authentication logic to provide meaningful error messages and guide users in case of authentication failures.

4

Testing and Evaluation

The note-taking app underwent a comprehensive testing and evaluation process to ensure its quality and usability. A systematic testing approach was followed, including unit testing to validate individual components and functions, integration testing to verify the interactions between different app components and Firebase services, and user testing involving the target audience to evaluate the app's usability and gather feedback. Additionally, performance evaluation measured the app's responsiveness and resource utilisation, while usability evaluation assessed the app's ease of use and overall user satisfaction. The testing and evaluation phase played a crucial role in identifying and addressing any issues, resulting in an improved and user-friendly note-taking app.

5

Conclusion

In conclusion, the development of the note-taking app using Flutter and Firebase has been a significant achievement.

By leveraging the capabilities of Flutter and Firebase, we were able to develop a robust and feature-rich app that offers a synchronised note-taking experience across multiple devices.

The implementation phase involved the utilisation of Flutter's cross-platform framework, which allowed for the development of a visually appealing and responsive user interface. Firebase's powerful backend services, including Firebase Authentication, Firebase Firestore, were seamlessly integrated to provide secure user authentication, real-time data synchronisation.

The note-taking app offers a range of features, including note creation, editing,deleting and account management.Firebase Firestore ensures that changes made to notes on one device are instantly reflected across all devices, ensuring a seamless and consistent user experience. User feedback and testing results have indicated a positive response to the simplicity, synchronisation capabilities, and intuitive interface of the app.

The project encountered various challenges, such as data synchronisation and integration between Flutter and Firebase. However, through careful planning, diligent testing, and leveraging the extensive documentation and community support available, these challenges were overcome, resulting in a stable and reliable application.

In conclusion, the note-taking app project has successfully achieved its objectives, delivering a high-quality application that meets the needs of users in the digital note-taking space. The project highlights the effectiveness of utilising Flutter and Firebase technologies to develop robust and user-friendly mobile applications. Moving forward, continuous improvement and further exploration of new features will contribute to the continued success of the note-taking app.

Overall, the note-taking app project has been a rewarding and valuable experience for the team, providing us with practical skills in mobile app development, collaboration, and problem-solving. We are proud of our accomplishments and look forward to future opportunities to enhance and expand the application's functionalities.

6

Recommendation

Future Improvements and Enhancements:

Implement rich text editing capabilities: Enhance the note creation and editing functionalities by incorporating rich text editing features, such as formatting options (bold, italics, bullet points).
Implement Search: Allow users to search their note with the matching keyword.
Folder implementation: Allow user to create a collection to store their different types of notes